Keywords: test-time reasoning • self-consistency • error localization • token efficiency

StepBack: Step-Level Error-Localized Resampling for Efficient Test-Time Reasoning

Localizing the First Error Step and Resampling Only Its Continuation Cuts Token Usage by Up to 41% at No Accuracy Cost

By Yang Ouyang, Jung-Eun Kim

arXiv:2611.08403 • cs.AI / cs.LG • NeurIPS 2026

Self-consistency generates many full chain-of-thought trajectories and takes a majority vote, but every chain runs to completion regardless of where the error occurs. StepBack localizes the most likely first error using lightweight signals and resamples only from that step onward, matching self-consistency accuracy on seven benchmarks while reducing token use by up to 41.26%.

AT A GLANCE

Problem: Self-consistency is expensive because every reasoning trajectory runs to completion, even when an error occurs early.

Why it matters: Test-time compute budgets are a primary cost driver for deployed LLM reasoning; a method that preserves accuracy at lower token cost directly reduces inference cost.

Method: Localize the first likely error in draft trajectories using NLL and PRM signals, then resample only the erroneous step and its continuation, finishing with majority voting.

Result: Matches SC accuracy on 7 benchmarks; cuts token usage by up to 41.26%.

Evidence: GSM8K, MATH, AIME, MultiArith, HotpotQA, 2WikiMultiHopQA, MuSiQue; Llama-3.1-8B-Instruct backbone.

The Big Picture

Chain-of-thought reasoning substantially improves LLM accuracy by externalizing intermediate steps. Self-consistency (SC) amplifies this further: by sampling many independent trajectories and taking a majority vote, SC cancels out reasoning errors that occur in different places on different runs. The cost is linear in the number of trajectories — a budget of 32 trajectories means 32× the tokens of a single greedy answer.

The key observation of StepBack is that this cost model is wasteful. Errors in a chain-of-thought trajectory are not uniformly distributed across steps. They tend to cluster in the early-to-middle portion of a chain, and once an error is made, subsequent steps correctly propagate it to a wrong final answer. Regenerating the entire trajectory from scratch discards all the correct prefix steps.

StepBack changes the unit of resampling from *full trajectory* to *erroneous step and its continuation*. By retaining the correct prefix, it recovers exactly the token saving wasted on correct steps in SC.

Why Existing Approaches Fall Short

Self-consistency (Wang et al., 2022) achieves its accuracy benefit by diversity of reasoning paths, not by quality filtering. It treats every trajectory uniformly and cannot distinguish high-quality from low-quality prefixes.

Best-of-N and verifier-guided methods re-rank complete trajectories after generation. They still generate all N full chains and then select the best; the selection step does not reduce the generation cost.

Speculative decoding accelerates token generation but does not reduce the *number* of tokens in the reasoning chain, only the wall-clock time. It is orthogonal to StepBack.

Adaptive Consistency reduces the number of full trajectories by stopping early when the answer distribution is sufficiently peaked. StepBack and Adaptive Consistency are orthogonal: StepBack shortens each trajectory, while Adaptive Consistency reduces the trajectory count. The two compose.

Core Mechanism

Given a draft set of m trajectories, StepBack splits each trajectory τ_i into atomic steps $\{s_i^1, s_i^2, \ldots, s_i^T\}$ using the model's natural delimiter tokens (newline, period, or explicit step markers).

For each step s_i^t, two signals are computed:

1. **NLL signal:** The mean negative log-likelihood of tokens in s_i^t under the model, $\mathrm{NLL}(s_i^t)$.
2. **PRM signal:** The process reward model score $\mathrm{PRM}(s_i^t \mid s_i^{<t})$, estimating the probability that all steps up to and including s_i^t are correct.

The anchor step $s_i^{a_i}$ is the earliest step satisfying:

$$\mathrm{NLL}(s_i^{a_i}) > \tau_{\mathrm{NLL}} \quad \text{or} \quad \mathrm{PRM}(s_i^{a_i} \mid s_i^{<a_i}) < \tau_{\mathrm{PRM}},$$

with $\tau_{\mathrm{NLL}} = 0.1$ and $\tau_{\mathrm{PRM}} = 0.05$. If no step is flagged, the trajectory is accepted as-is.

For each flagged trajectory, $k = 2$ fresh continuations are sampled from the anchor step onward, keeping the prefix $s_i^{<a_i}$ fixed. The final answers from all trajectories (original and resampled) are pooled and the plurality answer is returned.

Technical Formulation

Let $\mathcal{T} = \{\tau_1, \ldots, \tau_m\}$ be the initial set of $m = 16$ draft trajectories. For trajectory τ_i, the prefix up to step $a_i - 1$ is $p_i = s_i^1 \cdots s_i^{a_i - 1}$ and the original continuation is $c_i = s_i^{a_i} \cdots s_i^T$.

Resampled continuations are drawn from the conditional distribution:

$$\tilde{c}_i^j \sim p_\theta(\cdot \mid x, p_i), \quad j = 1, \ldots, k,$$

where x is the input question and p_θ is the LLM. The full answer set for majority voting is:

$$A = \{\text{ans}(\tau_i) : \tau_i \text{ not flagged}\} \cup \{\text{ans}(p_i \oplus \tilde{c}_i^j) : \tau_i \text{ flagged}, j = 1, \ldots, k\}$$

The returned answer is $\arg\max_a \sum_{a' \in A} \mathbf{1}[a' = a]$.

The expected token count per input is:

$$\mathbb{E}[\text{tokens}] = m \cdot \bar{L}_{\text{prefix}} + \alpha \cdot k \cdot \bar{L}_{\text{suffix}},$$

where α is the fraction of trajectories flagged and $\bar{L}_{\text{prefix}}, \bar{L}_{\text{suffix}}$ are mean prefix and suffix lengths. When errors cluster early ($\bar{L}_{\text{prefix}}$ small), the saving is largest.

Learning or Inference Procedure

StepBack is a pure inference-time method; no model parameters are modified. The required inputs are:

- A base LLM for draft generation and resampling.
- A process reward model (PRM) trained to score individual reasoning steps.
- Two scalar thresholds τ_{NLL} and τ_{PRM}, tuned on a small validation split.

The PRM used in experiments is a Llama-3.1-8B model fine-tuned on step-level correctness labels from the PRM800K dataset.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is: on the seven benchmarks tested, StepBack's accuracy is not statistically significantly different from SC at the same sampling budget of $m = 32$ effective trajectories. The token-reduction claim (up to 41.26%) is an empirical upper bound observed on the MATH benchmark; the reduction is smaller on benchmarks where errors occur later in trajectories.

Experimental Findings

- **Accuracy:** Statistically indistinguishable from SC on all 7 benchmarks (95% confidence intervals overlap in all cases).
- **Token reduction:** 20–41% fewer tokens than SC across benchmarks; largest on MATH (41.26%) and 2WikiMultiHopQA (38.1%).
- **Composition with Adaptive Consistency:** Step-Back + Adaptive Consistency achieves $> 41\%$ token reduction at equal or slightly higher accuracy than either method alone.
- **Threshold sensitivity:** Grid search over $\tau_{\text{NLL}} \in \{0.05, 0.1, 0.2\}$ and $\tau_{\text{PRM}} \in \{0.02, 0.05, 0.1\}$ shows accuracy within 1% of the reported configuration across the full grid.

Ablations and Interpretation

NLL-only vs. PRM-only vs. combined: NLL alone reduces tokens by 28%; PRM alone by 31%; combined by 37% on average. The two signals flag different steps on roughly 20% of trajectories, confirming complementarity.

Resampling k: $k = 1$ achieves 90% of the token savings at a 1–2% accuracy drop; $k = 3$ shows no further accuracy gain over $k = 2$ while increasing cost.

Step granularity: Fine-grained steps (sentence level) outperform coarse steps (paragraph level) by 1.3% average accuracy; the localized error is more precisely identified at finer granularity.

PRM quality: Replacing the trained PRM with a zero-shot LLM self-evaluation of step correctness reduces accuracy by 2.1% and token savings by 8%, confirming the importance of a dedicated PRM.

Reference

Yang Ouyang, Jung-Eun Kim. **StepBack: Step-Level Error-Localized Resampling for Efficient Test-Time Reasoning.** arXiv:2611.08403. NeurIPS 2026. https://arxiv.org/abs/2611.08403

Keywords: evolutionary optimization • latent agents • program synthesis • scientific computing

SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization

Agents That Reason in Activation Space Rather Than Text Evaluate 5–13× More Programs Per Hour on Mathematical and Scientific Discovery

By Qile Jiang, George Em Karniadakis

arXiv:2611.08483 • cs.AI / cs.NE • Brown University

LLM-guided evolutionary search is accurate but slow because each candidate requires a full chain-of-thought forward pass. SwarmEvolve replaces text-decoding agents with latent agents that share KV caches and reason in activation space, evaluating 5–13× more programs per hour than state-of-the-art frameworks and reaching equivalent solution quality in 3–8 minutes versus 60 minutes for competing systems.

AT A GLANCE

Problem: LLM-guided evolutionary search is accurate but too slow for large-scale discovery; each round requires full chain-of-thought for every candidate.

Why it matters: Scientific computing and mathematical discovery require evaluating hundreds of program variants per experiment; throughput is a hard bottleneck.

Method: 10 swarms of 45 latent agents share KV caches and reason for 40 latent steps without decoding to text; a lightweight decoding head maps top agent states to program tokens.

Result: 5–13× more programs per hour; solution quality reached in 3–8 min vs. 60 min for text-based frameworks.

Evidence: 21 problems spanning symbolic math and PDE solver kernel optimization; baselines: OpenEvolve, ShinkaEvolve, CodeEvolve.

The Big Picture

Program evolution with LLMs (OpenEvolve, FunSearch, ShinkaEvolve) has produced surprising discoveries in combinatorics and numerical computing by using LLMs as intelligent mutation operators: given a population of programs and their fitness scores, the LLM proposes new variants that improve on the best candidates. The approach works because LLMs encode prior knowledge about program structure, algorithm design, and mathematical transformations that random mutation cannot access.

The fundamental bottleneck is throughput. Generating a new program candidate with chain-of-thought reasoning costs one full LLM forward pass with a long context window. At 450 tokens per second on typical hardware, a 1,000-token chain-of-thought allows only 27 candidates per minute per GPU. Scientific computing problems may need thousands of candidates before a significant improvement is found.

SwarmEvolve attacks this bottleneck by eliminating the decoding step from most of the computation. Agents in SwarmEvolve do not produce natural language — they produce latent representations and hand them to a compact decoding head. The decoding head is cheap, while the latent reasoning is parallelized across 450 agents per round.

Why Existing Approaches Fall Short

Text-based evolutionary LLM frameworks (OpenEvolve, CodeEvolve) generate each candidate as a full text completion. This is flexible but slow: each candidate occupies the full generation path of the LLM.

Parallel sampling with temperature can explore multiple candidates simultaneously, but each sample is independent and cannot condition on what other concurrent samples are producing, limiting diversity within a batch.

Speculative decoding speeds up text generation but does not reduce the number of distinct candidates that can be evaluated per unit time, since it optimizes throughput on a single sequence.

Smaller surrogate models are faster but lack the reasoning quality of large LLMs, leading to lower-quality proposals and slower convergence in solution quality.

Core Mechanism: Latent Agents with KV Sharing

Each agent is a frozen LLM conditioned on a shared prefix KV cache covering the problem description and global best solution. The agent then runs 40 autoregressive steps *without* applying the unembedding matrix: no token is produced, and no text is decoded. Instead, the final hidden state $\mathbf{h}_i \in \mathbb{R}^d$ encodes the agent's "proposal" in activation space.

Agents in the same swarm share a common prefix KV cache $\mathcal{K}$, which eliminates redundant computation for the shared context and implicitly exposes each agent to the best solution found so far. Agents have different random seeds (different initial noise vectors appended to the shared prefix), inducing diversity within the swarm.

A lightweight decoding head $D_\phi : \mathbb{R}^d \to \mathcal{V}^L$ maps the hidden states of the top-r agents (by cosine similarity to the fitness-weighted centroid of the swarm) to program

token sequences of length at most $L = 200$ tokens.

Technical Formulation

Let $\mathcal{S} = \{S_1, \ldots, S_K\}$ denote $K = 10$ swarms, each containing $N = 45$ agents. The shared prefix for swarm S_k is:

$$\mathcal{K}_k = \text{KVCache}\big([\text{problem}, \mathbf{x}_k^*]\big),$$

where $\mathbf{x}_k^*$ is the globally best program found in rounds $1, \ldots, t-1$ of swarm k.

Each agent $a_{k,i}$ produces a hidden state:

$$\mathbf{h}_{k,i} = f_\theta\big(\mathbf{z}_{k,i}, \mathcal{K}_k\big) \in \mathbb{R}^d,$$

where $\mathbf{z}_{k,i} \sim \mathcal{N}(0, \sigma^2 I)$ is the agent-specific noise vector and f_θ runs 40 latent steps.

The swarm centroid weighted by fitness is:

$$\bar{\mathbf{h}}_k = \frac{\sum_{i=1}^{N} w_i \mathbf{h}_{k,i}}{\sum_{i=1}^{N} w_i}, \quad w_i = \text{softmax}(F_{k,i}/T),$$

where $F_{k,i}$ is the previous-round fitness of agent $a_{k,i}$ and T is a temperature parameter. The top-$r = 5$ agents (by cosine similarity to $\bar{\mathbf{h}}_k$) are decoded by D_ϕ.

Learning or Inference Procedure

D_ϕ is a 2-layer MLP trained offline on (hidden-state, program) pairs extracted from text-based evolutionary runs. At inference time, D_ϕ is frozen; SwarmEvolve requires no online training.

The evolutionary loop per round:

1. Update all K prefix caches with current best solutions.
2. Run all $K \times N = 450$ agents in parallel (each 40 latent steps) on a single A100 node.
3. Select top-r agents per swarm using the fitness-weighted cosine similarity score.
4. Decode selected agents' hidden states to programs via D_ϕ.
5. Execute decoded programs in a sandboxed environment.
6. Update global best solutions for each swarm.

What the Guarantee Says

No formal convergence guarantee is provided. The empirical claim is throughput superiority on 21 specific problems. The best-known solutions to 11 of the 21 problems (symbolic math subset) have formal certificates; SwarmEvolve recovers certified-correct solutions for 9 of these 11 within the allocated time budget of 2 GPU-hours.

Experimental Findings

- **Throughput:** 5–13× more programs per hour than all three baselines on all 21 problems.
- **Time-to-solution:** Median 3–8 minutes to reach the quality of the best competing solution; baselines require 60 minutes on the same hardware.
- **Solution quality:** Highest average score across 21 problems; wins 17 of 21 head-to-head comparisons vs. the best single baseline per problem.
- **Diversity:** The 10-swarm configuration produces 1.9× more distinct program families per hour than a 450-agent single-swarm variant, confirming the value of swarm isolation for population diversity.

Ablations and Interpretation

Latent steps: 40 latent steps is near-optimal; 20 steps reduces quality by 4.3 average points; 80 steps shows no improvement while doubling per-round time.

Swarm count: 10 swarms outperform 5 swarms by 2.7 average points and 1 swarm by 7.1 points, confirming that swarm diversity is the main quality driver.

KV sharing: Removing the shared prefix KV cache (agents start from scratch each round) reduces throughput by 2.3× and quality by 3.2 points, since agents lose access to the current best solution.

Decoding head quality: Replacing D_ϕ with a linear probe reduces program validity (fraction of programs that compile and run) from 94% to 71%, confirming the need for a non-linear decoder.

Reference

Qile Jiang, George Em Karniadakis. **SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization.** arXiv:2611.08483. https://arxiv.org/abs/2611.08483

Keywords: LLM inference • quantization • memory hierarchy • KV cache

StrataFlux: Memory-Hierarchy-Native Low-Bit LLM Inference

Using Host DRAM and NVMe SSD as Precision Strata Recovers Near-Full Accuracy at 2-Bit Weight Quantization Without Extra GPU Memory

By Jake Hyun, Ziyu Zhao, Jae-sun Seo, Mohamed S. Abdelfattah

arXiv:2611.08363 • cs.LG / cs.DC • submitted 5 Nov 2026

StrataFlux recovers accuracy lost to 2–4-bit quantization by treating the full memory hierarchy — GPU HBM, host DRAM, and NVMe SSD — as a stratified precision resource. On 2-bit Llama-3.1-8B-Instruct, WikiText-2 perplexity drops from 9.46 to 8.40 at 95% base throughput; on 2-bit Qwen3-8B, AIME 2024/2025 accuracy rises from 11.0% to 47.1% with no additional GPU memory.

AT A GLANCE

Problem: 2-bit quantization drastically reduces GPU memory but collapses accuracy; existing mixed-precision fixes keep all higher-precision weights on-GPU, negating the memory benefit.

Why it matters: Enabling 2-bit inference at acceptable accuracy would halve peak GPU memory for many deployments, making 70B+ models accessible on single-GPU servers.

Method: Stratified sensitivity profiling assigns weight layers to GPU HBM (2-bit), DRAM (4-bit), or NVMe SSD (8-bit); a prefetch scheduler hides tier-B/C transfer latency behind GPU compute.

Result: 2-bit Llama-3.1-8B perplexity 8.40 (vs. 9.46 baseline); 2-bit Qwen3-8B AIME accuracy 47.1% (vs. 11.0%).

Evidence: WikiText-2, AIME 2024/2025, 128K long-context; compared against 2-bit RTN, GPTQ, AWQ; RTX 5090.

The Big Picture

Large language model inference is bounded by memory bandwidth, not compute. Weights must be loaded from GPU memory on every forward pass, and peak GPU memory sets the maximum model size that fits on a single device. Quantization reduces both: 2-bit weights need 16× less memory than FP32 and consume less bandwidth per layer.

The problem is accuracy. At 2 bits, the quantization error per weight is large enough to damage the model's reasoning ability, particularly on tasks that require precise recall of numerical facts or long multi-step chains. Prior solutions (GPTQ, AWQ, SpQR) identify sensitive weights and store them at higher precision *on the GPU*, but this cancels the memory benefit for the sensitive fraction.

StrataFlux reframes the question: the GPU is not the only memory in a modern server. Host DRAM is 8–32× larger and 5–10× slower; NVMe SSD is 10–100× larger still. The sensitive weights that need 4- or 8-bit precision can live in DRAM or SSD and be streamed to GPU only when needed for a specific layer, overlapped with the GPU computation on the previous layer.

Why Existing Approaches Fall Short

Uniform quantization (RTN, 2-bit) is fast and memory-efficient but loses too much accuracy in sensitive layers, especially attention projections and early embedding layers.

Mixed-precision on-GPU (GPTQ, AWQ, SpQR) keeps sensitive weights at 4–8 bits on the GPU. This recovers accuracy but requires extra GPU memory proportional to the number of sensitive parameters, partially defeating the purpose of quantization.

CPU offloading (FlexGen, DejaVu) moves entire layers to CPU for latency-insensitive serving but introduces large latency spikes when layers are paged back in, making throughput unpredictable.

StrataFlux differs from CPU offloading in two ways: (1) it offloads only the *higher-precision copies* of sensitive layers, not the full layer; the base 2-bit copy always stays on GPU for fast path; and (2) a prefetch scheduler computes the access pattern ahead of time and hides DMA transfers behind GPU computation.

Core Mechanism: Three-Tier Precision

Stratum A (GPU HBM): The base quantized weights of all layers at the target bit-width (2–4 bits). Always resident; used for fast-path inference.

Stratum B (host DRAM): 4-bit copies of the top-k% most sensitive layers, identified by calibration. Paged in one block ahead of the layer it corrects.

Stratum C (NVMe SSD): 8-bit copies of an extremely sensitive subset (typically the first and last 2 transformer blocks, plus any layer where calibration error exceeds a threshold). Paged in asynchronously via direct DMA.

For a layer assigned to stratum B or C, the forward pass combines:

$$\mathbf{y} = W_{\text{base}}\mathbf{x} + \Delta W_{\text{tier}}\mathbf{x},$$

where W_{base} is the 2-bit on-GPU weight and $\Delta W_{\text{tier}} =$

$W_{\text{tier}} - W_{\text{base}}$ is the correction loaded from DRAM or SSD. This residual form ensures that if the DMA transfer has not completed (e.g., SSD latency spike), the base path produces an approximate result instead of blocking.

Technical Formulation

Sensitivity of layer l is estimated by the Fisher trace:

$$S_l = \operatorname{tr}(\mathbf{H}_l) \approx \frac{1}{|C|} \sum_{x \in C} \left\| W_l x - \hat{W}_l x \right\|_2^2,$$

where C is a calibration corpus, W_l is the full-precision weight, and $\hat{W}_l$ is its 2-bit quantized version. Layers are ranked by S_l and assigned to strata by two thresholds θ_B and θ_C:

$$\begin{aligned} l &\in \text{Stratum A if } S_l \le \theta_B, \\ l &\in \text{Stratum B if } \theta_B < S_l \le \theta_C, \\ l &\in \text{Stratum C if } S_l > \theta_C. \end{aligned}$$

Thresholds θ_B and θ_C are set by a memory budget constraint: the DRAM and SSD footprints are fixed by the available hardware, and layers are sorted by S_l until the budget is consumed.

Learning or Inference Procedure

StrataFlux requires no fine-tuning; it modifies only the inference memory layout and access pattern. The workflow is:

1. Run calibration pass on 128 calibration sequences to compute S_l for all layers.
2. Assign layers to strata based on thresholds.
3. Quantize and place stratum-B copies in DRAM, stratum-C in SSD.
4. At inference time, the prefetch scheduler issues DMA requests for the next 3 stratum-B layers and 1 stratum-C layer ahead of execution, overlapping transfers with GPU computation.

What the Guarantee Says

No theoretical accuracy bound is provided. The empirical claim is Pareto-dominance over existing mixed-precision methods on the two primary benchmarks: StrataFlux achieves lower perplexity at lower GPU memory than any configuration of GPTQ or AWQ tested. No latency bound under SSD-latency spikes is given; the base path ensures correctness but not accuracy.

Experimental Findings

- **Perplexity:** 2-bit Llama-3.1-8B-Instruct WikiText-2 PPL 8.40 (StrataFlux) vs. 9.46 (2-bit RTN base); 95% base throughput on RTX 5090.
- **Reasoning:** 2-bit Qwen3-8B AIME 2024/2025 accuracy 47.1% (StrataFlux) vs. 11.0% (2-bit RTN); GPU memory unchanged.
- **Long-context:** 3-bit weights + 2-bit KV cache at 128K context matches vLLM 4-bit AWQ + 8-bit KV perplexity and speed at half the GPU memory.
- **Throughput:** Prefetch scheduler achieves 95% of base 2-bit throughput by hiding DRAM latency; SSD accesses visible only when DRAM budget is exhausted.

Ablations and Interpretation

Stratum-C necessity: Removing stratum C (SSD tier) and assigning all stratum-C layers to stratum B (DRAM at 4-bit) increases DRAM footprint by 18% at no accuracy change, confirming that the SSD tier covers only a small but critical fraction of sensitive layers.

Residual form: Replacing the residual correction ΔW_{tier} with a full replacement (always load W_{tier} on this path) reduces throughput by 12% because the base 2-bit GPU path is always bypassed, even when DMA is on time.

Calibration size: 32 calibration sequences achieve 94% of the sensitivity ordering quality of 512 sequences; 128 sequences reach 99%.

Model size: Relative perplexity improvement is larger for 8B-class models (1.1 PPL points) than for 70B-class models (0.4 PPL points), reflecting that smaller models are more sensitive to 2-bit quantization and benefit more from stratum correction.

Reference

Jake Hyun, Ziyu Zhao, Jae-sun Seo, Mohamed S. Abdelfattah. **StrataFlux: Memory-Hierarchy-Native Low-Bit LLM Inference.** arXiv:2611.08363. https://arxiv.org/abs/2611.08363

Keywords: reinforcement learning • reward machines • vision-language models • non-Markovian tasks

RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines

An LLM Builds the Finite Automaton; a VLM Grounds Its Propositions; the Agent Earns Credit for History-Dependent Task Progress

By Roko Parac, Christos Ziakas, Leo Ardon

arXiv:2611.08308 • cs.AI • submitted 6 Nov 2026

Most VLM-based reward methods assume that reward depends only on the current observation. RM-VLM handles tasks that require history-dependent credit by having an LLM construct a reward machine from a text description and a VLM ground each proposition in image observations. Evaluated on grid navigation, robotic manipulation, and 3D object collection, RM-VLM consistently outperforms existing VLM-reward methods on task success.

AT A GLANCE

Problem: VLM reward methods assume Markovian tasks; real tasks often require ordered sub-goals whose credit depends on history.

Why it matters: Without history-aware reward, RL agents fail to distinguish correct-order completions from incorrect-order ones and cannot learn ordered, multi-step tasks reliably.

Method: LLM decomposes task description into propositions and builds a reward machine (finite automaton); VLM grounds propositions in image observations; VLM value estimator provides dense shaping.

Result: Consistently higher task success than all baselines across three task domains.

Evidence: Grid-world navigation, robotic manipulation sequence, 3D object collection; baselines: VLM-RM, CLIP-reward, TAMER+VLM.

The Big Picture

Reinforcement learning agents historically required handcrafted reward functions that encode both the goal and the intermediate milestones. VLM-based reward models automate this by scoring image observations against a natural-language goal description: an agent that moves closer to the goal receives a higher reward from the VLM, without any manual reward engineering.

This works well for Markovian goals where the reward depends only on the current state. It breaks for *ordered* tasks: "pick up the red block, then place it in the green zone, then open the door." A VLM reward that evaluates only the current image cannot distinguish whether the red block was placed *before* or *after* the green zone interaction. An agent that learns to shortcut to the door without completing earlier sub-goals will receive high reward on the final state.

Reward machines (Icarte et al., 2018) are the standard formalism for non-Markovian task structure: a finite automaton tracks which sub-goals have been achieved and transitions on logical propositions evaluated in the environment. RM-VLM combines the LLM's ability to decompose natural-language task descriptions with the VLM's ability to ground propositions in images.

Why Existing Approaches Fall Short

VLM scalar reward (VLM-RM) scores the current image against the full task description. It cannot distinguish sub-goal ordering and provides identical reward to agents that complete sub-goals in any order.

CLIP-reward embeds both image and text goal in CLIP space and uses cosine similarity as reward. It has the same Markovian limitation and additionally fails to track incremental progress toward structured sub-goals.

TAMER with VLM feedback uses intermittent human-style feedback modeled by a VLM. The feedback is episodic and noisy; it does not provide dense, automaton-guided credit for individual transitions.

Handcrafted reward machines (prior RM work) require a domain expert to specify the propositions and automaton structure manually — the bottleneck RM-VLM eliminates.

Core Mechanism

Step 1: Reward machine construction. The task description $\mathcal{T}$ is passed to an LLM with a structured prompt requesting:

- A set of Boolean propositions $\mathcal{P} = \{p_1, \ldots, p_n\}$ over environment entities.
- A finite automaton $M = (Q, q_0, q_f, \delta, \rho)$ where $\delta : Q \times 2^{\mathcal{P}} \to Q$ is the transition function and $\rho : Q \times Q \to \mathbb{R}$ assigns reward to transitions.

The LLM outputs M as a JSON object, which is validated and compiled.

Step 2: VLM grounding. At each step, the VLM receives the current image observation o_t and each proposition p_i, returning $P(p_i{=}\text{true} \mid o_t)$. The resulting Boolean vector advances M's state through δ, triggering the reward $\rho(q_t, q_{t+1})$ on any transition.

Step 3: Dense reward shaping. To alleviate sparse

rewards in long-horizon tasks, a VLM-based value estimator V_{VLM} scores the image against the proposition set of each pending transition, producing a shaped bonus:

$$r_t^+ = \lambda \sum_{p_i \in \text{pending}(q_t)} P(p_i{=}\text{true} \mid o_t),$$

where λ is a shaping coefficient and $\text{pending}(q_t)$ is the set of propositions required to advance from q_t.

Technical Formulation

The reward machine $M = (Q, q_0, q_f, \delta, \rho)$ defines a non-Markovian MDP $\mathcal{M} = (S, A, T, R_M)$ where the total reward at step t is:

$$R_M(s_t, a_t) = \rho(q_t, q_{t+1}) + r_t^+,$$

with $q_{t+1} = \delta(q_t, L(o_t))$ and $L(o_t) \in 2^{\mathcal{P}}$ the labelling function implemented by the VLM. The augmented state (s_t, q_t) is Markovian in $\mathcal{M}$, so standard policy-gradient methods (PPO, SAC) apply.

The labelling function uses a threshold $\tau = 0.5$:

$$L(o_t) = \{p_i : P(p_i{=}\text{true} \mid o_t) \geq \tau\}.$$

Learning or Inference Procedure

1. Submit task description $\mathcal{T}$ to LLM to obtain M as JSON.
2. Validate M (check that q_f is reachable, no duplicate states, all propositions well-formed).
3. Initialize policy π_θ with PPO; use (s_t, q_t) as the augmented state.
4. At each environment step, query VLM for $L(o_t)$, advance M, and compute $R_M(s_t, a_t)$.
5. Train until convergence (5M environment steps per domain).

No fine-tuning of the LLM or VLM is performed; both are used frozen.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is consistent task-success superiority over all baselines in all three domains tested. No sample complexity bound is given; the 5M-step budget is task-specific and may not be sufficient for harder domains.

Experimental Findings

- **Grid-world navigation:** RM-VLM task success 87% vs. VLM-RM 61%, CLIP-reward 54%, TAMER+VLM 70%.
- **Robotic manipulation sequence:** RM-VLM 74% vs. VLM-RM 38%, CLIP 29%, TAMER+VLM 51%; largest gap because out-of-order completions are highest in this domain.
- **3D object collection:** RM-VLM 81% vs. VLM-RM 58%.
- **Dense shaping:** Removing the shaped bonus r_t^+ reduces robotic manipulation success from 74% to 52%, confirming its role in long-horizon credit propagation.

Ablations and Interpretation

LLM construction errors: On 15% of randomly sampled task descriptions, the LLM produces a machine with at least one missing transition or unreachable state. These cases are caught by validation and trigger a retry; the retry resolves 90% of errors.

VLM grounding accuracy: Replacing the VLM labelling function with ground-truth oracle labels improves manipulation success from 74% to 88%, establishing an upper bound from grounding accuracy limitations.

Threshold τ: Grid search over $\tau \in \{0.3, 0.5, 0.7\}$ shows $\tau = 0.5$ optimal across all three domains; $\tau = 0.3$ causes false-positive transitions; $\tau = 0.7$ misses valid ones.

Shaping coefficient λ: $\lambda = 0.1$ is optimal; larger values cause the agent to maximize the shaped bonus rather than completing transitions, a known reward-shaping pathology.

Reference

Roko Parac, Christos Ziakas, Leo Ardon. **RM-VLM: From Natural-Language Task Descriptions to Vision-Grounded Reward Machines.** arXiv:2611.08308. https://arxiv.org/abs/2611.08308

Keywords: medical AI • graph memory • clinical dialogue • patient state tracking

MedGraph: Maintaining Patient State with Graph Memory for Long-Term Medical Dialogue

Three-Layer Hierarchical Graph Memory Eliminates Outdated State, Cross-Patient Contamination, and Missing Safety Facts, Gaining 10.5 Points on MedMemoryBench

By Donghua Zhang, Zizheng Wang, Ze Dong, Yiding Feng, Siyu Qian

arXiv:2611.08433 • cs.CL / cs.AI • submitted 6 Nov 2026

Medical dialogue assistants fail in longitudinal settings because flat memory systems accumulate outdated states, mix facts across patients, and silently drop safety information. MedGraph introduces a three-layer hierarchical graph memory that makes clinical state transitions explicit and unconditionally prepends safety facts to every query. On MedMemoryBench it exceeds the strongest of five baselines by 10.4–10.5 percentage points with both GPT and Qwen backbones.

AT A GLANCE

Problem: Flat memory systems for clinical dialogue produce outdated state, cross-patient contamination, and silently missed safety facts across multi-visit conversations.

Why it matters: LLMs used as clinical assistants in longitudinal care must maintain accurate, up-to-date patient state; errors can cause harmful medication or diagnosis decisions.

Method: Three-layer memory: raw transcripts (L1), a typed clinical fact graph with temporal validity tags (L2), and a patient profile unconditionally attached to every query (L3).

Result: 10.4–10.5 pp above the strongest of five baselines on MedMemoryBench with GPT and Qwen backbones.

Evidence: MedMemoryBench (multi-visit clinical dialogues); baselines: flat vector, sliding window, RAG, Zep, MemoryOS.

The Big Picture

Large language models deployed as clinical dialogue assistants need to maintain accurate knowledge of a patient's medical state across many visits over weeks or months. Existing memory approaches — flat vector stores, sliding-window context, retrieval-augmented generation — treat memory as a bag of text fragments. This fails in three recurring ways identified by the authors through systematic error analysis of five baseline systems on longitudinal medical dialogues:

1. **Outdated state:** A resolved symptom is reported as currently active because an older transcript fragment scores higher in retrieval than the later discharge note.
2. **Cross-patient contamination:** Facts from a different patient's earlier session pollute the current patient's context when embedding similarity overrides patient identity.
3. **Missing safety facts:** A known allergy or contraindication is not retrieved because it was not in the top-k most similar fragments for the current query.

MedGraph introduces structure that makes all three failures impossible by design: typed temporal graph updates prevent outdated state; strict per-patient graph isolation prevents contamination; and mandatory safety-block attachment prevents silent omission.

Why Existing Approaches Fall Short

Flat vector memory embeds all clinical utterances and retrieves the top-k most similar. It does not distinguish whether a fact is currently active or resolved, and similarity can produce facts from other patients if patients share medical vocabulary.

Sliding-window context keeps the most recent n tokens. Long-term facts (a years-old allergy, a chronic condition diagnosed at first visit) fall out of the window and are lost.

Retrieval-augmented generation (RAG) retrieves relevant documents at query time. It improves recall of historical facts but cannot distinguish the current clinical state from historical states when multiple conflicting observations exist in the corpus.

Zep (entity memory) maintains entity states but uses flat entity records without typed temporal transitions; it conflates active and resolved states.

MemoryOS provides structured memory with hierarchical layers but does not specialize for clinical entity types or enforce mandatory safety-fact prepending.

Core Mechanism: Three-Layer Memory

Layer 1 — Raw dialogues. Full session transcripts are stored verbatim, enabling exact retrieval of clinical reasoning chains and direct-quotation evidence when needed. This layer is searched by traditional BM25 retrieval for specific historical passages.

Layer 2 — Clinical fact graph. A typed knowledge graph over entities (symptoms, diagnoses, medications, lab results, procedures) with typed relations (has-symptom, prescribed, diagnosed-with, resolved, contraindicated-with). Each entity node carries a temporal validity tag: *active*, *resolved*, or *historical*. Graph construction and update are performed by a clinical NLP extraction pipeline applied to each new session.

Layer 3 — Patient profile (safety block). A compact structured record per patient containing: recorded allergies, contraindications, chronic conditions, emergency contacts, and medication alerts. This block is *unconditionally prepended* to every query context, bypassing the retrieval mechanism entirely. No fact in this block can be missed because retrieval does not decide whether it is included.

Graph isolation. Each patient has a fully separate graph instance and a separate patient profile. Cross-patient contamination is architecturally impossible.

Technical Formulation

Let $\mathcal{G}_p = (V_p, E_p)$ be the clinical fact graph for patient p, where each node $v \in V_p$ is a clinical entity with attributes $(\text{type}, \text{value}, \text{validity})$ and each edge $e = (v_i, v_j, r)$ is a typed relation.

When a new session transcript D_t arrives for patient p, the extraction pipeline produces a set of entity mentions $\hat{V}_t$ and relation mentions $\hat{E}_t$. The update procedure applies type-specific rules:

$$\text{if } v \in \hat{V}_t \text{ and } v \notin V_p: \quad \text{add } v \text{ with validity} = \textit{active},$$

$$\text{if } v \in \hat{V}_t \text{ and } v \in V_p \text{ and ``resolved'' signal:} \quad \text{set validity} \rightarrow \textit{resolved},$$

$$\text{if } v \in \hat{V}_t \text{ and } v \in V_p \text{ and no state change:} \quad \text{update timestamp only}.$$

The "resolved" signal is detected by a small classifier trained on clinical resolution phrases ("symptom resolved", "discontinued", "normal finding", etc.).

At query time, the context assembled for the LLM backbone is:

$$\mathcal{C} = \underbrace{\text{Profile}_p}_{\text{L3, mandatory}} \oplus \underbrace{\text{Graph}_p^{\text{active}}}_{\text{L2, active facts}} \oplus \underbrace{\text{RAG}(\mathcal{D}, q)}_{\text{L1, retrieved}},$$

where $\text{Graph}_p^{\text{active}}$ is the subgraph of currently active entities serialized as structured text, and $\text{RAG}(\mathcal{D}, q)$ is BM25-retrieved transcript fragments relevant to query q.

Learning or Inference Procedure

MedGraph uses no end-to-end learning. Its components are:

- A clinical NER/RE pipeline (pre-trained, fine-tuned on MIMIC-IV notes) for entity and relation extraction.
- A resolution-signal classifier (a small BERT-based model fine-tuned on clinical resolution annotation).
- An arbitrary LLM backbone (tested with GPT-4o-mini and Qwen2.5-72B) for the final answer generation.

The NLP pipeline and classifier are frozen at inference time.

What the Guarantee Says

No formal guarantee is claimed. The empirical claim is superior clinical accuracy across all five baselines on MedMemoryBench with both LLM backbones. The error analysis identifies three failure modes and shows that MedGraph's design eliminates each by construction; the reduction in those specific error types is verified per-category.

Experimental Findings

- **Overall accuracy (GPT-4o-mini):** MedGraph +10.5 pp above best baseline (Zep).
- **Overall accuracy (Qwen2.5-72B):** MedGraph +10.4 pp above best baseline.
- **Outdated-state errors:** Reduced by 68% relative to Zep, confirming typed temporal update effectiveness.
- **Safety-fact recall:** 100% safety-fact recall (by mandatory prepending), vs. 73–82% for retrieval-based baselines.
- **Cross-patient contamination rate:** 0% (by graph isolation), vs. 3–7% for vector-memory baselines.

Ablations and Interpretation

Without mandatory safety block: Dropping the unconditional prepend and relying on retrieval for safety facts reduces safety-fact recall from 100% to 79%, confirming that retrieval alone is insufficient.

Without typed temporal updates: Using simple entity overwrite (new mention always replaces old) increases outdated-state errors by 3.2×, as resolved symptoms return when re-mentioned in a summary context.

Without graph isolation: Merging all patients into a single graph increases cross-patient contamination to 4.1%, confirming that per-patient isolation is necessary.

L1 retrieval necessity: Removing L1 (raw transcript retrieval) reduces overall accuracy by 4.3 pp, showing that verbatim historical passages provide useful evidence beyond what the structured graph captures.

Reference

Donghua Zhang, Zizheng Wang, Ze Dong, Yiding Feng, Siyu Qian. **MedGraph: Maintaining Patient State**

with Graph Memory for Long-Term Medical Dialogue. arXiv:2611.08433. https://arxiv.org/abs/2611.08433